

Internet Storage Systems
Programming Assignment #1 - Team 4
Cecilia Durst, Francoise Hayek, Wilbert Hernandez

Software Design:

1. Server.java

a. public class Server

- i. A client-side socket, server socket, data input stream, and data output stream are initialized
- ii. public Server(int port)
 1. The server socket is declared using the port parameter and host IP address.
 2. The client-side socket listens for a connection through the server socket.
 3. The data output stream is connected to the client-side socket and a "Hello!" message is sent.
 4. The data input stream is connected to the client-side socket, and an empty string m is created to hold client messages.
 5. While m does not equal "bye," m listens for a client message. Upon receipt, the regular expression "[a-zA-Z]+" is matched to the message to check if it is alphabetical. If so, the message is set to upper case using String.toUpperCase() and sent back to the client. If the message is not alphabetical, then the server sends a message asking the client to send an alphabetical message.
 6. After the message "bye" is received, the client socket, input stream, and output stream are closed.

b. public static void main(String args[])

- i. If there is one command-line argument, it is attempted to be used as the port number for Server.
- ii. If there are no command-line arguments, port number 6000 is automatically passed into Server.
- iii. If there are more than one command-line arguments, the user is asked to enter zero or one.

2. Client.java

a. public class Client

- i. A client-side socket, data input stream from the terminal, data input stream from the socket, and data output stream to the socket are initialized.
- ii. public Client(String addr, int port)

1. The client-side socket is defined using the IP address (addr) and port number (port) parameters.
2. The data stream input for the terminal is defined
3. The data stream output for the socket is defined
4. The data stream input for the socket is defined
5. The client listens for the 'Hello!' message from the server, signifying that a connection was established
6. A while loop initiates that runs until the client-side user inputs 'bye'
7. The client-side user is prompted for a message
8. Once a message is given, a timer is started.
9. The given message is passed to the server via the socket
10. Upon receiving a response from the server, the timer is stopped and the resultant elapsed time is recorded as a data point
11. Once a 'bye' message is received from the client-side user, 'bye' is sent to the server and the connection is closed.
12. computeRTTStatistics is called, with the recorded data points
- iii. private void computeRTTStatistics
 1. The recorded round-trip time data points (if any) are used to calculate the minimum, maximum, mean, and standard deviation
 2. The calculated information is presented to the client-side user
- b. public static void main(String[] args)
 - i. If there are two command-line arguments, they are attempted to be used as the IP address and port number for Server.
 - ii. If there are no command-line arguments, IP address '127.0.0.1' (local host) and port number 6000 is automatically passed into Client.
 - iii. If there are one or more than two command-line arguments, the user is asked to enter none or two.

Timer Implementation and Performance Measurement

1. Design and Implementation

The timer functionality is implemented on the client-side to measure the Round-Trip Time (RTT) for a message exchange between the client and server. RTT is defined as the time taken:

1. From when the client sends a message to the server.
2. To when the client receives the capitalized response back.

To measure this accurately, we used `System.currentTimeMillis()` in Java, which records timestamps in milliseconds.

2. Timer Process Flow

1. Client starts the timer immediately before sending the message.
2. Client sends the message to the server.
3. Server processes the message:
 - If the input consists only of alphabets, it converts the string to uppercase and sends it back.
 - If the input contains non-alphabet characters, the server sends an error message, prompting the client to re-enter a valid string.
4. Client receives the response and stops the timer.
5. Client calculates the RTT as: $RTT = (EndTime) - (StartTime)$
6. The process is repeated five times, and the following statistics are calculated:
 - Minimum RTT
 - Maximum RTT
 - Mean RTT
 - Standard Deviation of RTT

3. RTT Measurement Code Snippet

```
long startTime = System.currentTimeMillis();    // Start Timer
out.writeUTF(m);                                // Send message to the server
m = serverIn.readUTF();                         // Receive response from the server
long endTime = System.currentTimeMillis();     // Stop Timer
long rtt = endTime - startTime;                // Calculate RTT
rttTimes.add(rtt);
```

4. RTT Results and Analysis

Through multiple test cases on a local machine, we observed:

- The first message typically has a slightly higher RTT (~9-10 ms) due to TCP handshake overhead.
- Subsequent messages show RTT values ranging between 5-8 ms, as no new connection is established.
- Standard deviation remains low, indicating a stable and predictable response time.

Example Output (Across Separate Machines):

RTT Statistics:

Min RTT: 5 ms

Max RTT: 14 ms

Mean RTT: 7.92 ms

Standard Deviation: 2.28 ms

Observations and Lessons Learned

- The first message exhibits a higher RTT due to connection setup.
- RTT is near zero on a local machine, but if the server were on a different machine, network latency would increase.
- The timer accurately captures RTT and provides meaningful statistics.
- Setting up sockets on both client and server sides is fundamental. We learned how to open connections, send and receive data, and how to configure settings like the IP address and port.
- Handling exceptions (like `IOException` or `UnknownHostException`) is crucial. It ensures that the program can gracefully deal with unexpected network issues, making our application more robust.
- Closing all streams and sockets properly after communication ends prevents resource leaks. This is important for maintaining system stability, especially in longer-running or more complex applications.
- Measuring RTT helps in understanding the responsiveness of our network connection. This simple performance metric can guide in optimizing communication or diagnosing latency issues.